Exp 8

Aim :- To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory :-

1. Understanding Ganache as a Local Blockchain Platform

Ganache is a personal Ethereum blockchain used for development and testing. It simulates a real blockchain environment but runs locally on your computer. This allows developers to deploy contracts, test transactions, and debug without spending real cryptocurrency.

Key characteristics of Ganache :

- It provides a set of pre-funded test accounts.
- It runs a local Ethereum node instantly without syncing.
- It allows inspection of blocks, transactions, gas usage, and contract events.
- It is ideal for rapid prototyping and debugging before deploying to public testnets or mainnet.

Ganache is part of the Truffle Suite and is widely used in Ethereum development workflows. It acts as a controlled environment where developers can repeatedly deploy and test smart contracts without cost.

---

2. Role of MetaMask in Smart Contract Deployment

MetaMask is a browser-based Ethereum wallet and gateway to blockchain networks. It manages private keys, signs transactions, and connects decentralized applications to networks such as :

- Local blockchain (Ganache)
- Public testnets (Goerli, Sepolia)
- Ethereum mainnet

When using Ganache, MetaMask is configured to connect to the local RPC server (usually http://127.0.0.1:7545 (127.0.0.1 in Bing)). The accounts generated by Ganache can be imported into MetaMask using their private keys. This allows MetaMask to sign and send transactions to the local blockchain.

MetaMask acts as the bridge between Remix IDE and Ganache by authorizing deployments and contract interactions.

---

3. Remix IDE as a Smart Contract Development Environment

Remix IDE is a browser-based platform for writing, compiling, deploying, and testing Solidity smart contracts. It provides :

- A Solidity editor with syntax highlighting
- A compiler for generating bytecode and ABI
- Deployment tools that connect to MetaMask
- A user interface for interacting with deployed contracts
- Debugging and analysis tools

Remix can deploy contracts to :

- JavaScript VM (in-browser blockchain)
- Injected Provider (MetaMask)
- External networks (Ganache, testnets, mainnet)

When MetaMask is connected to Ganache, Remix uses MetaMask as the transaction signer and Ganache as the execution environment.

---

4. Deploying a Smart Contract Using Ganache, MetaMask, and Remix

The deployment process involves several conceptual steps :

1. Writing the smart contract
   A Solidity contract is created defining state variables, functions, and logic.

2. Compiling the contract
   Remix compiles the Solidity code into EVM bytecode and generates the ABI (Application Binary Interface).
   The ABI defines how external applications interact with the contract.

3. Connecting Remix to MetaMask
   Remix uses the Injected Provider option to route deployment transactions through MetaMask.

4. MetaMask connecting to Ganache
   MetaMask is configured to use Ganache's RPC endpoint.
   When Remix sends a deployment request, MetaMask signs the transaction using the selected account.

5. Deployment to Ganache
   Ganache receives the signed transaction, mines a block instantly, and stores the contract on the local blockchain. A contract address is generated.

6. Interacting with the contract
   Remix displays the deployed contract's functions.
   Each function call becomes a transaction (if it modifies state) or a call (if it is read-only).
   Ganache logs all transactions, gas usage, and events for inspection.

This workflow allows developers to test contract logic thoroughly before deploying to real networks.

---

5. Example of a Simple Solidity Contract

A minimal example of a contract that can be deployed using this workflow :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
   uint public value;

   function setValue(uint _value) public {
      value = _value;
   }
}
```

This contract demonstrates :

- A state variable stored on the blockchain
- A function that modifies state (requires a transaction)
- A public getter function automatically generated by Solidity

---

6. Summary of the Complete Workflow

1. Install and run Ganache to create a local blockchain.
2. Open MetaMask and configure it to connect to Ganache's RPC network.
3. Import Ganache accounts into MetaMask using private keys.
4. Write and compile the Solidity contract in Remix IDE.
5. Select Injected Provider (MetaMask) in Remix.
6. Deploy the contract; MetaMask signs the transaction.
7. Ganache mines the transaction and deploys the contract.

8. Interact with the contract through Remix while Ganache logs all activity.

Code :-

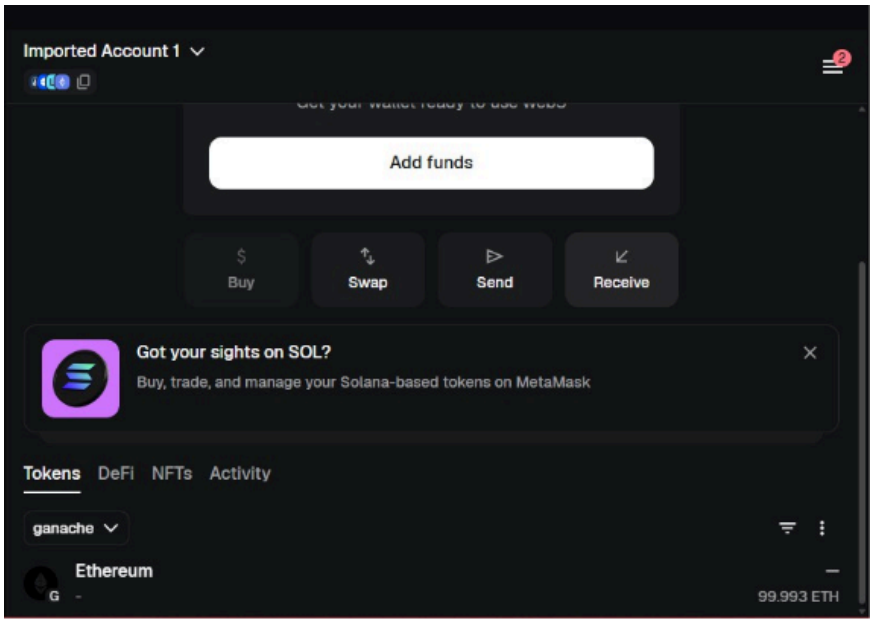
```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.0;

contract CharityDonation {
	address public owner; uint public totalDonations;
	struct Donation {
		address donor; uint amount; uint timestamp;
		}

Donation[] public donations;

event DonationReceived(address donor, uint amount); event FundsWithdrawn(address owner,
uint amount);

constructor() {
	owner = msg.sender;
	}

// Function to donate Ether function donate() public payable {
	require(msg.value > 0, "Donation must be greater than zero");

	donations.push(Donation(msg.sender, msg.value, block.timestamp)); totalDonations +=
	msg.value;

	emit DonationReceived(msg.sender, msg.value);
	}

// Function to check contract balance
function getBalance() public view returns (uint) {
	return address(this).balance;
	}

// Only owner can withdraw funds function withdraw(uint amount) public {
	require(msg.sender == owner, "Only owner can withdraw funds"); require(amount <=
	address(this).balance, "Insufficient balance");

	payable(owner).transfer(amount);
	emit FundsWithdrawn(owner, amount);
	}
```

```
// View number of donations
function getDonationCount() public view returns (uint) {
        return donations.length;
        }
}
```

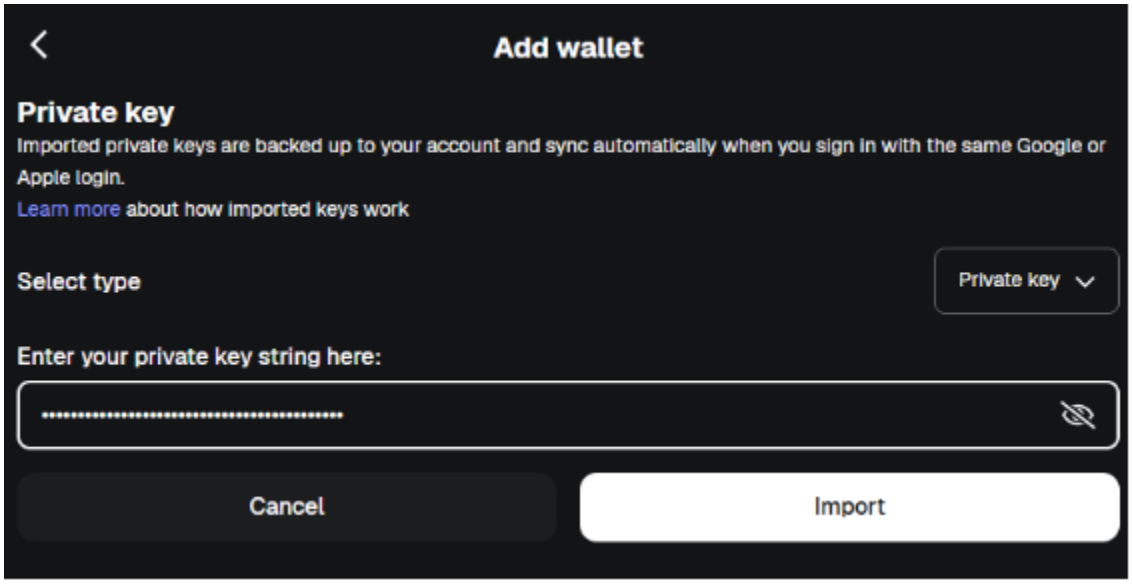
Metamask

<u>Screenshots</u> :-

Adding funds for testing :

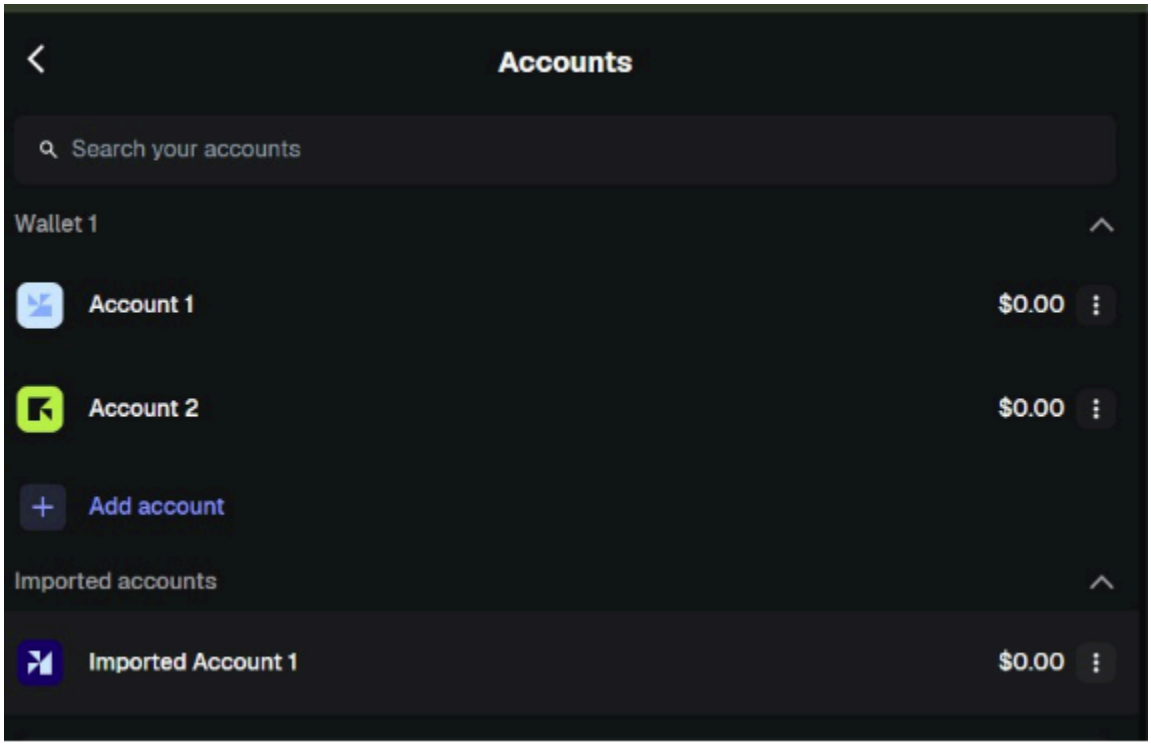


Entering wallet keys :

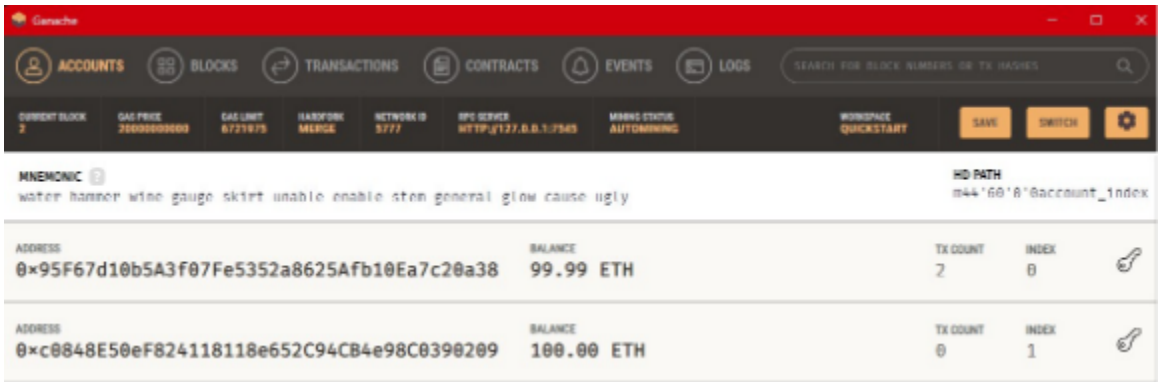


D20A 59 Jai Talreja Blockchain Lab

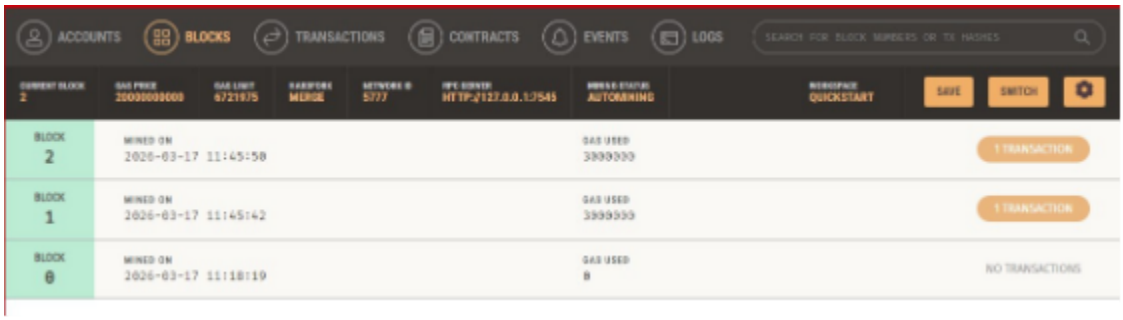
Accounts for the funds to be added to for testing :



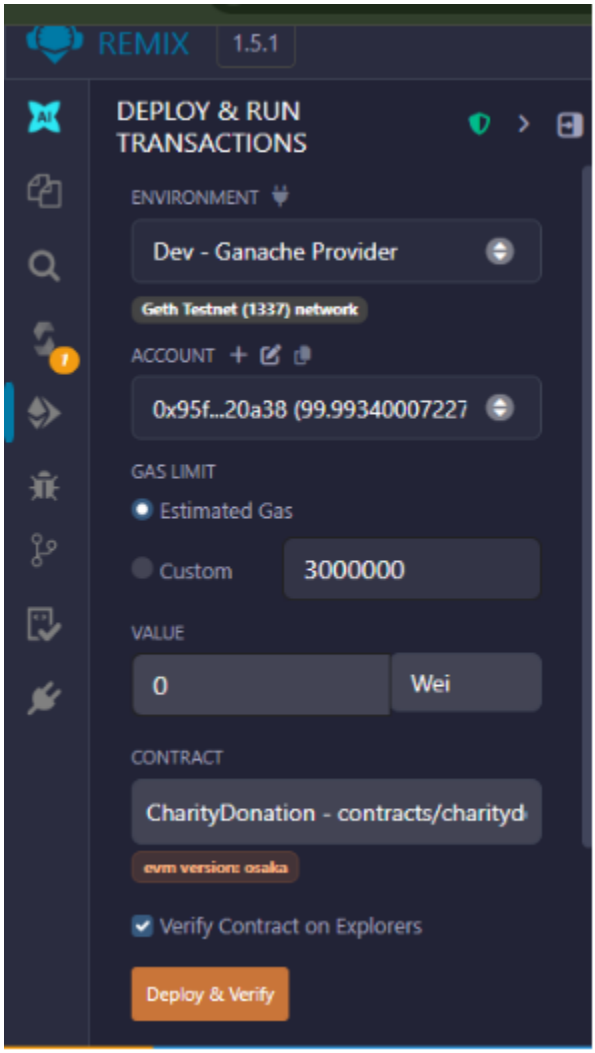
Transaction pending :



Transaction completed :



Contract deployed and executed :



<u>Conclusion</u> :-

This experiment demonstrated the implementation of a blockchain-based environment using Ganache, MetaMask, and Remix IDE. A smart contract was successfully created, deployed, and tested using Ganache accounts integrated with MetaMask. The experiment also highlighted the potential of blockchain technology in enhancing charity donation systems by ensuring transparency, security, and traceability of transactions. Overall, it provided practical knowledge of executing and monitoring blockchain transactions in a controlled environment.